

Lab – Azure Container Apps: Deploying the Application Container

To begin with the Lab

Summary

In this lab, we deploy a **.NET application container** to **Azure Container Apps** using an image stored in **Azure Container Registry (ACR)**. A new **Container Apps Environment** is created, the container image (Version 2) is selected, ingress is enabled for internet access, and the application is exposed through HTTP.

Understanding

Azure Container Apps is a fully managed service for running containerized applications without managing virtual machines or Kubernetes clusters. Applications are deployed from container images stored in registries such as Azure Container Registry (ACR).

Before deploying a container app, Azure requires a **Container Apps Environment**, which provides the infrastructure where one or more container apps run. The environment also defines networking, monitoring, and workload profiles.

Since the application is a .NET web application listening on **port 8080**, Azure Container Apps automatically receives HTTP requests on **port 80** and forwards them to **port 8080** inside the container. By enabling **Ingress**, the application becomes accessible over the public internet.

Example

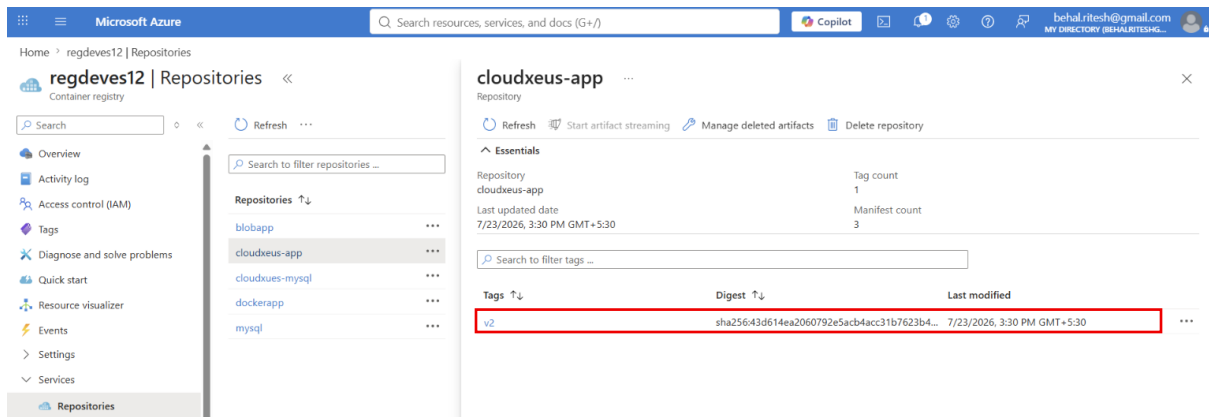
Suppose you have a .NET web application stored as **cloudxeusapp:v2** in Azure Container Registry.

- Azure Container Apps pulls the image from ACR.
 - The container starts and listens on **port 8080**.
 - Users access the application using the Azure Container App URL.
 - Azure automatically forwards requests from **port 80** to **port 8080** inside the container.
-

Lab Steps

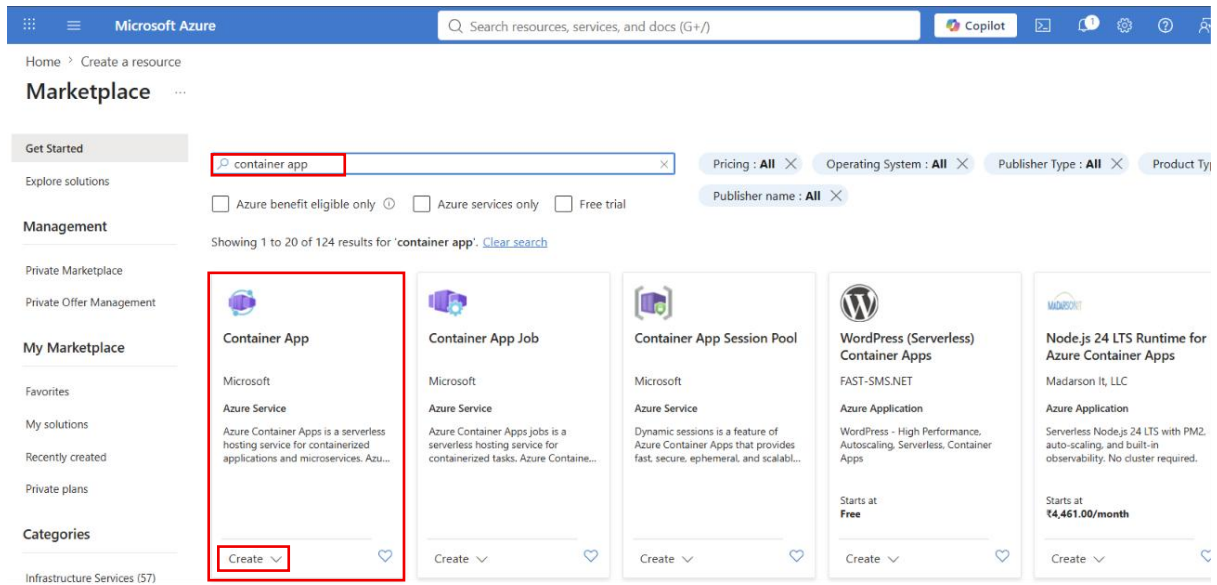
Step 1: Verify the Container Image

- Open **Azure Container Registry**.
- Ensure **Version 2 (v2)** of the application image exists.
- Confirm the image is ready for deployment.



Step 2: Open Azure Container Apps

- Sign in to the **Azure Portal**.
- Search for **Container Apps**.
- Click **Create**.



Step 3: Configure Basic Settings

- Select the **Resource Group**.
- Enter the **Container App Name** (for example: cloudxeus-app).
- Set Deployment Source to **Container Image**.

Home > Create a resource > Marketplace

Create Container app ...

Basics Container Ingress Tags Review + create

Create a containerized app and run it on a serverless platform—without managing cloud infrastructure. [Quickstart guide](#)

Project details

Select a subscription to manage resource creation and costs, and a resource group to organize all your resources for this deployment.

Subscription * Visual Studio Enterprise Subscription

Resource group * Webapp
[Create new resource group](#)

Container app name * cloudeus-app

Optimize for Azure Functions ☐

☒ Built-in support and autoscaling for Azure Functions (requires image compatible with Functions). [How to run functions with your container app](#)

Deployment source * ☒ Container image
Bring your own container registry or build a container from a Dockerfile.

☐ Source code or artifact
Build and deploy your code without using a Dockerfile.

Step 4: Select the Container Image

- Open the **Container** tab.
- Select your **Azure Container Registry (ACR)**.
- Choose the application image.
- Select **Version 2 (v2)**.
- Set **Authentication Type** to **Secrets**.
- Azure automatically uses the registry username and password.

Home

Create Container app ...

Basics **Container** Ingress Tags Review + create

Select a quickstart image for your container, or deselect quickstart image to use an existing container.

Use quickstart image ☐

Container details

Name * cloudeus-webapp

Image source ☒ Azure Container Registry
☐ Docker Hub or other registries

Subscription * Visual Studio Enterprise Subscription

Registry * regdevs12.azurecr.io

Image * cloudeus-app

Image tag * v2

Registry authentication

Authentication type ☐ Managed identity ☒ Secrets

The ACR username and password will be automatically retrieved

Step 5: Configure the Container

- Leave the **Workload Profile** unchanged.

- Select **.NET** as the Development Stack.
- Keep the remaining container settings as default.

Home

Create Container app ...

Development stack-specific features

When you select a specific development stack, you get additional features tailored to that stack—optimizing Container Apps to perform for your unique settings.

Development stack .NET ▼

[Customize .NET features for your app](#)

[Learn more](#) about .NET-specific features

Container resource allocation

Choose the workload profile for this app. You can adjust the CPU and memory allocation for this app up to the workload profile limit. [Learn more](#)

Workload profile * Consumption - Up to 4 vCPUs, 8 Gib memory ▼

GPU ☒

CPU and memory * 0.5 CPU cores, 1 Gi memory ▼

Environment variables

Name ▼	Value ▼	Delete
Enter name	Enter value	

Step 6: Configure Ingress

- Enable **Ingress**.
- Allow traffic from **Anywhere (Public Internet)**.
- Select **HTTP**.
- Leave the **Transport Protocol** as **Automatic**.
- Set the **Target Port** to **8080**.

Note:

- The application listens on **port 8080** inside the container.
- Azure Container Apps receives requests on **port 80** and automatically forwards them to **port 8080**.

Home

Create Container app ...

Basics Container Ingress Tags Review + create

Application ingress settings

Enable ingress for applications that need an HTTP or TCP endpoint.

Ingress  ☒

Ingress traffic ☐ Limited to Container Apps Environment
Select this option if you want to restrict traffic to this container app from within the Container App Environment

☒ Accepting traffic from anywhere
Select this option if you want to allow traffic to this container app from anywhere

Ingress type  ☒ HTTP

☐ TCP

Transport 

Insecure connections ☐

Target port 

Session affinity  ☐

 Additional TCP ports

Step 7: Review and Create

- Click **Review + Create**.
- Validate all settings.

Home

Create Container app ...

Basics Container Ingress **Tags** Review + create

Tags are name/value pairs that enable you to categorize resources and view consolidated billing by applying the same tag to multiple resources and resource groups.

Note that if you create tags and then change resource settings on other tabs, your tags will be automatically updated.

Name	Value	Resource
	:	

Review + create

< Previous

Next : Review + create >

- Click **Create**.

Home

Create Container app ...

✓ Passed

Basics Container Ingress Tags **Review + create**

Project details

Subscription	Visual Studio Enterprise Subscription
Resource group	DefaultResourceGroup-EUS
Name	cloudxeus-webapp

Container Apps Environment (New)

Region	eastus
Container Apps Environment	managedEnvironment-DefaultResource-bcce
Log Analytics workspace (New)	workspacedefaultresourcegroupeusabc2
Virtual network	Default
Zone redundancy	Disabled

Container

Name	cloudxeus-webapp
Image source	Azure Container Registry
Registry	regdeves12.azurecr.io
Image	cloudxeus-app
Image tag	v2

Create

< Previous

Next

[Download a template for automation](#)

Step 8: Wait for Deployment

- Azure first creates the **Container Apps Environment**.
- Azure then deploys the **Container App**.
- Wait until the deployment completes successfully.

Home

 **Microsoft.App-ContainerApp-Portal-1261cd0d-be99** | Overview ...

x x «  Delete  Cancel  Redeploy  Download  Refresh

 Overview

 Inputs

 Outputs

 Template

✓ **Your deployment is complete**

 Deployment name : Microsoft.App-ContainerApp-Portal-1261cd0d-be99
Subscription : [Visual Studio Enterprise Subscription](#)
Resource group : [DefaultResourceGroup-EUS](#)

Start time : 7/23/2026, 4:28:01 PM
Correlation ID : 2b228b66-6703-47f3-8308-d740b02c561d

> Deployment details

✓ Next steps

[Go to resource](#)

Give feedback

 [Tell us about your experience with deployment](#)

Step 9: Verify the Deployment

- Open the deployed **Container App**.
- Copy the **Application URL**.

Overview

- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Resource visualizer
- Application

Essentials

Resource group (move)	: DefaultResourceGroup-EUS
Status	: Running
Location (move)	: East US
Subscription (move)	: Visual Studio Enterprise Subscription
Subscription ID	: 294539c9-c109-4db8-b785-0043716c0b11
Aspire Dashboard	: Not yet active (set up)

Application Url	: https://cloudxeus-webapp.yellowhill-e631480b.eastus.azurecontainerapps.io
Container Apps Environ...	: managedEnvironment-DefaultResource-bcce
Environment type	: Workload profiles
Log Analytics	: workspaceDefaultResourceGroupEUSabc2
Development stack	: .NET (manage)

- Open the URL in a browser.
- Verify that the .NET web application loads successfully.



Welcome

Learn about [building Web apps with ASP.NET Core](#).